CS285 HW1 — Flow Matching Action-Chunking Policy Report

Oct 14, 2026 · Shakthibala Sivagami Balamurugan

Result

The flow matching policy reaches a final evaluation reward of **0.817** on Push-T, above the 0.7 target. It passes the target at step 50,000 (0.748) and is still improving at the end. The MSE policy from Part 2 reached 0.629 with the same network size and training budget.

Reward is the mean, over 100 evaluation episodes, of each episode's maximum reward. The full run (75,600 steps, including 8 evaluations) took 17 min 18 s on CPU. Run logs and rollout videos: WandB run lys577me.

Training curves

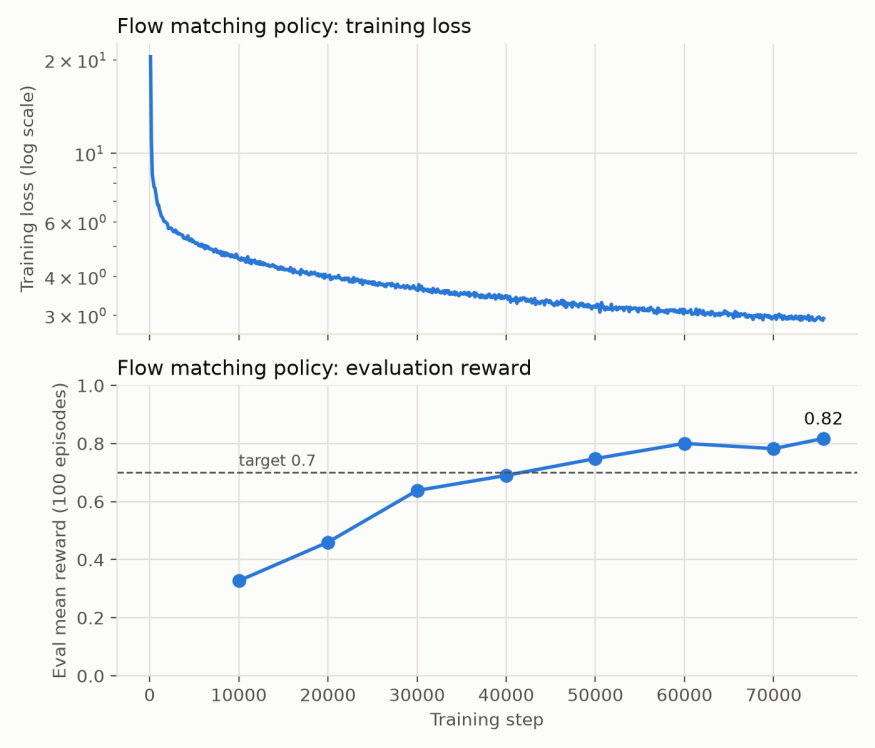


Figure 1: Training loss (log scale) and evaluation mean reward against training step for the flow matching policy

Training loss falls from 20.5 at step 100 to 2.93 at the end. It stays well above zero because the target $A_t - A_{t,0}$ contains random noise the network cannot predict, so this loss is not comparable to the MSE loss. Evaluation reward climbs steadily, from 0.33

at step 10,000 to 0.80 at step 60,000 and 0.82 at the end. MSE instead levelled off around 0.63 after step 50,000. Plotted from the run's logged metrics with matplotlib (`plot_curves.py`), not from WandB screenshots.

Flow matching vs. MSE: behavior

MSE either solves an episode cleanly or gets stuck; flow matching keeps moving, recovers far more often, but rarely settles. To check what the videos suggested, both final checkpoints were re-run on the same 100 evaluation seeds:

Final checkpoint, eval seeds 0–99	MSE	Flow matching
Mean reward	0.629	**0.818**
Fully solved (coverage > 95%)	7	**17**
Nearly solved (reward ≥ 0.8)	53	**78**
Clear failures (reward < 0.3)	22	**8**
Action change inside / between chunks	7.3 / 18.5 px	8.4 / 16.7 px

Flow sampling is random, so its re-run (0.818) differs slightly from the logged 0.817; MSE is deterministic and matches exactly.

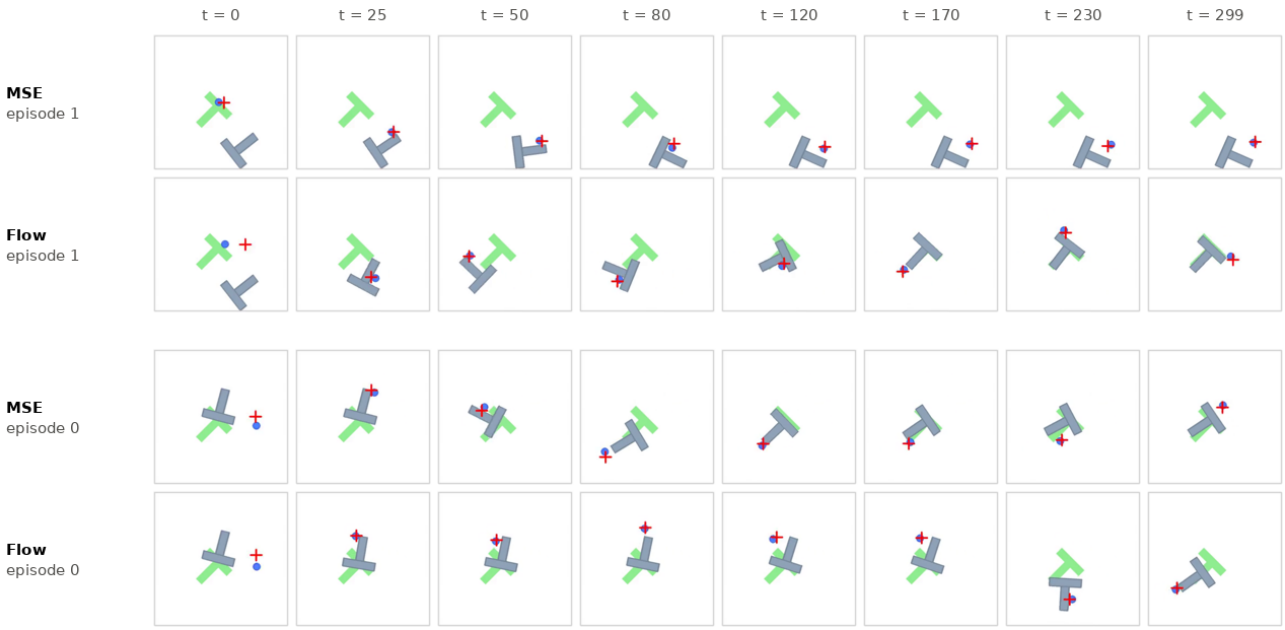


Figure 2: Frames from the final-checkpoint rollout videos. Each row pair starts from the same initial state.

- **MSE freezes when the right move is ambiguous.** In episode 1 it pushes the T toward the bottom right, then parks beside it from about step 80 to the end, and the reward stays at 0. Averaging two opposite pushes gives almost no motion, and because the policy is deterministic, the same state produces the same chunk every time, so it never escapes.
- **Flow matching recovers.** From the same start it drags the T back to the goal and rotates it into place. Each chunk is a fresh sample, so the agent tries different

contact points instead of repeating one move. This is why its clear failures drop from 22 to 8.

- **Flow matching can waver.** In episode 0 it sits on top of a wrongly rotated T from about step 25 to step 170 without making progress, then knocks the T away and starts over. Consecutive chunks can come from different strategies, which shows up as hesitation.
- **Flow matching rarely settles.** Near the goal it keeps making small corrections instead of holding still, so only 1 of its 40 recorded videos ends early, against 6 of 40 for MSE. Over 100 seeds it still fully solves more episodes (17 vs. 7).
- **Both policies jump at chunk boundaries.** The action changes about twice as much between chunks as inside one, because each chunk runs open-loop for 8 steps before re-planning.

MLP architecture

The velocity network uses the same MLP as the MSE policy: 3 hidden layers of 256 units with ReLU, then a linear output layer. Its input is the 5-D normalized state, the noisy chunk $A_{t,\tau}$ flattened to 16 numbers, and the flow timestep τ as one raw scalar, 22 numbers in total. Its 16 outputs are reshaped to an 8 × 2 velocity for the whole chunk.

Layer	Input → output	Activation	Parameters
Hidden 1	22 → 256	ReLU	5,888
Hidden 2	256 → 256	ReLU	65,792
Hidden 3	256 → 256	ReLU	65,792
Output	256 → 16 (reshaped to 8 × 2)	none (linear)	4,112
Total			**141,584**

There is no time embedding, dropout, normalization layer or weight decay.

Training setup

The loss is Eq. 2: the squared L2 error between the predicted velocity and the straight-line velocity from noise to the expert chunk, averaged over the batch.

$$\mathcal{L}_{FM}(\theta) = \frac{1}{B}\sum_{j=1}^{B}\left\| v_\theta\left(o_t^{(j)}, A_{t,\tau}^{(j)}, \tau^{(j)}\right) - \left(A_t^{(j)} - A_{t,0}^{(j)}\right)\right\|_2^2, \qquad A_{t,\tau} = \tau A_t + (1-\tau)A_{t,0}$$

At inference, a chunk is generated by Euler integration (Eq. 3) from $A_{t,0} \sim \mathcal{N}(0, I)$:

$$A_{t,\tau+1/n} = A_{t,\tau} + \tfrac{1}{n}\, v_\theta(o_t, A_{t,\tau}, \tau), \qquad \tau = 0, \tfrac{1}{n}, \ldots, \tfrac{n-1}{n}$$

Setting	Value
Data	Push-T, 206 expert episodes, 25,650 transitions → 24,208 (state, 8-action chunk) windows
Normalization	per-feature z-score of states and actions (dataset mean and std)
Noise and timestep	new $A_{t,0} \sim \mathcal{N}(0, I)$ and one $\tau \sim U(0, 1)$ per sample, drawn every batch
Integration	Euler, $n = 10$ steps
Optimizer	Adam, learning rate 3e-4, no weight decay
Batch size	128
Training length	400 epochs = 75,600 steps
Speed-up	`torch.compile` on the loss and its backward pass
Evaluation	every 10,000 steps and at the end: 100 episodes, chunks run open-loop, 5 rollout videos to WandB
Seed	42

All hyperparameters are the starter-code defaults, the same as the MSE run; no sweep was run.